

Lista de Atividade Prova 3 – AEDS II

Questões de implementação de algoritmos

Feito por: Gabriel Ferreira Pereira

Esta versão contém apenas questões de implementação. Foram removidas as questões de desenhar árvores, provar, refutar ou apenas interpretar conceitos.

Balanceamento

1) Contar nós não balanceados pelo critério AVL

Considere uma árvore binária não necessariamente balanceada. Implemente o método `contarNosNaoBalanceados()`, que deve retornar a quantidade de nós cujo fator de balanceamento não atende ao critério AVL. Não adicione novos atributos às classes. Faça também a análise de complexidade.

```
class Arvore {
    private No raiz;

    public int contarNosNaoBalanceados() {
        // IMPLEMENTAR
        return 0;
    }
}

class No {
    public int elemento;
    public No esq, dir;
}
```

2) Balanceamento local em AVL

Implemente o método `balancear(No no)` de uma árvore AVL. Considere que os métodos de rotação já existem. O método deve decidir entre rotação simples e dupla usando o fator de balanceamento do nó atual e do filho apropriado. Faça também a análise de complexidade.

```
class No {
    int elemento, altura;
    No esq, dir;
}

class AVL {
    private int getAltura(No no) {
        return (no == null) ? -1 : no.altura;
    }
}
```

```

private int fator(No no) {
    return (no == null) ? 0 : getAltura(no.dir) - getAltura(no.esq);
}

private No rotacionarEsq(No no) { return no; }
private No rotacionarDir(No no) { return no; }

public No balancear(No no) {
    // IMPLEMENTAR
    return no;
}
}

```

3) Inserção em AVL

Implemente o método `inserir(int x)` em uma árvore AVL. A inserção deve manter a propriedade de árvore binária de busca, atualizar as alturas dos nós e chamar o balanceamento quando necessário. Faça também a análise de complexidade.

```

class AVLInsercao {
    private No raiz;

    public void inserir(int x) {
        // IMPLEMENTAR
    }

    private No inserir(int x, No i) {
        // IMPLEMENTAR
        return i;
    }
}

```

Hash

4) Inserção em tabela hash com reserva

Implemente o método `inserir(String s)` em uma tabela hash que usa área primária e área de reserva linear para tratar colisões. A função hash é a soma dos códigos ASCII dos caracteres módulo `m1`. Faça também a análise de complexidade.

```

class HashReserva {
    String[] tabela;
    int m1, m2, reserva;

    public HashReserva(int m1, int m2) {
        this.m1 = m1;
        this.m2 = m2;
    }
}

```

```

        this.tabela = new String[m1 + m2];
        this.reserva = 0;
    }

    private int h(String s) {
        int soma = 0;
        for (int i = 0; i < s.length(); i++) soma += s.charAt(i);
        return soma % m1;
    }

    public boolean inserir(String s) {
        // IMPLEMENTAR
        return false;
    }
}

```

5) Pesquisa em hash com rehash linear

Implemente o método `pesquisar(int x)` em uma tabela hash aberta com rehash linear. Considere que posições vazias guardam `-1`. Faça a análise de complexidade média e de pior caso.

```

class HashLinear {
    int[] tabela;
    int m;

    private int h(int x) {
        return x % m;
    }

    public boolean pesquisar(int x) {
        // IMPLEMENTAR
        return false;
    }
}

```

6) Remoção lógica em hash linear

Implemente o método `remove(int x)` em uma tabela hash com rehash linear. Considere que `-1` representa posição vazia e `-2` representa posição removida. A remoção deve preservar a possibilidade de encontrar elementos que foram inseridos após colisões. Faça também a análise de complexidade.

```

class HashLinearRemocao {
    int[] tabela;
    int m;

    private int h(int x) {

```

```

        return x % m;
    }

    public boolean remover(int x) {
        // IMPLEMENTAR
        return false;
    }
}

```

Doidona e Estruturas Híbridas

7) Pesquisa na estrutura Doidona

Considere uma estrutura Doidona cujo primeiro nível contém uma árvore binária de busca com letras iniciais. Cada nó aponta para uma tabela T1. Em caso de colisão, a pesquisa deve continuar em T2 e depois em T3. Implemente o método `pesquisar(String palavra)`. Faça também a análise de complexidade.

```

class Doidona {
    private No raiz;

    public boolean pesquisar(String palavra) {
        // IMPLEMENTAR
        return false;
    }
}

class No {
    public char caractere;
    public T1 t1;
    public No esq, dir;
}

class T1 {
    public String[] palavras;
    public T2 t2;
}

class T2 {
    public String[] tabela;
    public T3 t3;
}

class T3 {
    public NoAB raiz;
    public Celula primeiro;
    public int hashT3(int x) { return x % 2; }
}

```

```

class NoAB {
    String palavra;
    NoAB esq, dir;
}

class Celula {
    String palavra;
    Celula prox;
}

```

8) Inserção em estrutura híbrida árvore + hash + lista

Implemente o método `inserir(String palavra)` em uma estrutura híbrida. O primeiro nível localiza a palavra pela primeira letra em uma árvore binária. O segundo nível usa uma tabela hash pelo último caractere. Se houver colisão, deve-se tentar um rehash. Se a colisão continuar, a palavra deve ser inserida em uma lista encadeada dentro de T2. Faça também a análise de complexidade.

```

class DicionarioHibrido {
    private NoLetra raiz;

    public void inserir(String palavra) {
        // IMPLEMENTAR
    }
}

class NoLetra {
    public char letra;
    public T1Hibrida t1;
    public NoLetra esq, dir;
}

class T1Hibrida {
    public String[] tabela;
    public T2Lista t2;
    public int m1;

    public int hashT1(char x) { return x % m1; }
    public int rehashT1(char x) { return (x + 1) % m1; }
}

class T2Lista {
    public CelulaLista[] tabela;
    public int m2;

    public int hashT2(int tamPalavra) { return tamPalavra % m2; }
}

class CelulaLista {
    public String palavra;
}

```

```
public CelulaLista prox;
}
```

Trie

9) Verificar prefixo e sufixo em trie

Implemente o método `existe(String inicio, String fim)` em uma trie. O método deve retornar `true` se existir pelo menos uma palavra completa que comece com `inicio` e termine com `fim`. Faça também a análise de complexidade.

```
class ArvoreTrie {
    NoTrie raiz;

    public boolean existe(String inicio, String fim) {
        // IMPLEMENTAR
        return false;
    }
}

class NoTrie {
    char elemento;
    CelulaTrie primeiro; // clula cabeca
    CelulaTrie ultimo;
    boolean fimPalavra;
}

class CelulaTrie {
    char elemento;
    CelulaTrie prox;
    NoTrie no;
}
```

10) Listar palavras com um dado prefixo

Implemente o método `mostrarComPrefixo(String pref)`, que imprime todas as palavras armazenadas em uma trie que começam com o prefixo informado. Primeiro localize o nó correspondente ao prefixo e depois percorra a subárvore. Faça também a análise de complexidade.

```
class Trie {
    private NoTrie raiz;

    public void mostrarComPrefixo(String pref) {
        // IMPLEMENTAR
    }
}
```

11) Inserção em trie com lista de filhos

Implemente o método `inserir(String palavra)` em uma trie cujos filhos de cada nó são armazenados em lista encadeada. O método deve criar novos nós quando necessário e marcar o fim da palavra corretamente. Faça também a análise de complexidade.

```
class TrieInsercao {
    private NoTrie raiz;

    public void inserir(String palavra) {
        // IMPLEMENTAR
    }
}
```

Patricia

12) Projeto e implementação de nó Patricia

Implemente a classe `NoPatricia`. A classe deve possuir atributos para armazenar o rótulo comprimido do arco, a marcação de fim de palavra e a estrutura usada para os filhos. Escolha uma das estratégias: vetor, árvore binária ou tabela hash.

```
class NoPatricia {
    // IMPLEMENTAR
}
```

13) Pesquisa em trie Patricia

Implemente o método `pesquisar(String palavra)` em uma trie Patricia. A pesquisa deve comparar os rótulos comprimidos dos arcos e avançar apenas quando o trecho da palavra corresponder ao rótulo do nó. Faça também a análise de complexidade.

```
class Patricia {
    private NoPatricia raiz;

    public boolean pesquisar(String palavra) {
        // IMPLEMENTAR
        return false;
    }
}
```